

Cloud Cost Optimizer – Prompt Documentation

This file documents the exact prompt used to build the Cloud Cost Optimizer AI Agent project. It shows how we used AI-assisted development, what instructions we gave, and how the entire system was designed and generated from a single structured prompt.

The Actual Prompt We Used to Build This Project

This is the complete prompt given to the AI to generate the entire Cloud Cost Optimizer system from scratch:

...

You are an expert Python developer and AI systems architect. Build me a complete, production-quality AI Agent project called "Cloud Cost Optimizer" from scratch.

PROJECT OVERVIEW

Build an autonomous AI Agent that:

1. Reads mock cloud billing CSV data (4 weeks of data) and auto- updates every 30 seconds and also when integrated with cloud platforms.
2. Detects idle/oversized resources using heuristics
3. Reasons about each finding using a local LLM (Ollama llama3)
4. Files prioritised GitHub Issues (P0/P1/P2) automatically
5. Sends live Discord alerts via webhook
6. Sends a formatted email report via SMTP
7. Generates and auto-opens a beautiful HTML dashboard with charts showing current scan + weekly trend analysis
8. Includes an embedded AI Assistant chatbot in the dashboard
9. Live watch mode – auto-refreshes dashboard every 30 seconds
10. Interactive CLI – ask cloud cost questions via --ask flag

This is a TOOL-USING AI AGENT – not a chatbot, not a web app. It runs as a single Python script: `python main.py`
Everything happens automatically after that one command.

FOLDER STRUCTURE

```
cloud-cost-optimizer/
├── data/
│   ├── cloud_billing_week1.csv
│   ├── cloud_billing_week2.csv
│   ├── cloud_billing_week3.csv
│   └── cloud_billing_week4.csv ← current week
├── src/
│   ├── __init__.py
│   ├── reader.py           ← reads and parses CSV files
│   ├── detector.py         ← heuristic flagging logic
│   ├── llm_analyzer.py     ← Ollama LLM reasoning
│   ├── github_issues.py    ← GitHub API integration
│   ├── discord_alert.py    ← Discord webhook alerts
│   └── email_report.py     ← SMTP email sending
```

```

├──┬─ html_report.py      ← HTML dashboard generator
├─┬─ output/
├─┬─┬─ dashboard.html   ← auto-generated, auto-opened
├─┬─ main.py
├─┬─ config.py
├─┬─ prompts.md
├─┬─ requirements.txt
├─┬─ README.md

```

MOCK DATA — ALL 4 CSV FILES

Columns: resource_id, type, region, cpu_avg_percent,
memory_avg_percent, cost_monthly_usd, last_active_days

Week 1: 6 resources, 1 flagged, ~\$120 waste
 Week 2: 7 resources, 2 flagged, ~\$240 waste
 Week 3: 8 resources, 3 flagged, ~\$405 waste
 Week 4: 8 resources, 3 flagged, ~\$525 waste (current)

Realistic data:

- EC2 instances: cpu varies 1-90%
- RDS databases: memory varies 1-85%
- S3 buckets: always 0% CPU
- Costs: EC2 \$150-220, RDS \$280-350, S3 \$80-100
- Flagged resources: <10% CPU and high cost

DETECTION RULES

- Idle: cpu_avg < 10% AND cost > \$50/month
- Oversized: cpu_avg < 20% AND cost > \$150/month
- Severity:
 - P0 Critical: saving > \$200/month OR cpu < 3%
 - P1 Warning: saving \$100-200/month OR cpu 3-7%
 - P2 Low: saving < \$100/month OR cpu 7-10%
- Savings estimation:
 - Stale >30 days: 90% of monthly cost
 - Low CPU/memory: 75% of monthly cost
 - Oversized: 55% of monthly cost
 - Other flagged: 35% of monthly cost

LLM ANALYZER — OLLAMA PROMPT

Use this exact prompt template for every flagged resource:

"You are a cloud cost optimization expert working for a DevOps team. Analyze this cloud resource and provide a cost-saving recommendation.

Resource Details:

- ID: {resource_id}
- Type: {type} (EC2/RDS/S3)
- Region: {region}
- CPU Usage: {cpu_avg_percent}% (average over 30 days)
- Memory Usage: {memory_avg_percent}%
- Monthly Cost: \${cost_monthly_usd}
- Days Since Last Active: {last_active_days}

Your job:

1. Determine if this resource is idle or oversized
2. Explain WHY in 1-2 sentences (plain English)
3. Give ONE specific recommended action
4. Estimate monthly savings if action is taken
5. Rate severity: P0 (critical), P1 (warning), P2 (low)

Respond ONLY in this exact JSON format, nothing else:

```
{
  'severity': 'P0',
  'reason': 'explanation here',
  'recommendation': 'specific action here',
  'estimated_saving': '$XXX/month',
  'confidence': 'high'
}"
```

- Handle Ollama connection errors gracefully
- Fallback to rule-based analyzer if Ollama is offline
- Add retry logic (3 attempts) for failed LLM calls

GITHUB ISSUES

- Auto-create labeled issues for every flagged resource
- Title format: [P0] server-03 is idle – save \$280/month
- Labels: cost-optimization + P0-critical/P1-warning/P2-low
- Label colors: P0=red #d73a4a, P1=yellow #e4e669, P2=green #0075ca
- Check for duplicates before filing
- Issue body in professional markdown with metrics table

DISCORD ALERTS

- Send rich embed alerts for P0 resources immediately
- Send full scan summary after all analysis completes
- Color coded: red P0, yellow P1, green P2
- Footer: "Cloud Cost Optimizer Agent • {timestamp}"

EMAIL REPORT

- Send fully formatted HTML email (not plain text)
- Inline CSS styling, dark theme matching dashboard
- Summary stats + findings table + GitHub issue links
- Subject: "Cloud Cost Report – {n} Issues Found | \${total} Savings"

HTML DASHBOARD

Build a BEAUTIFUL single-file offline HTML dashboard with:

Section 1 – Header with scan timestamp and Agent Active badge

Section 2 – 4 Stats Cards:

- Resources Scanned (blue)
- Issues Found (red)
- Monthly Savings \$ (green)
- GitHub Issues Filed (yellow)

Section 3 – Tabs: Summary, Charts, Resources, Trends

Section 4 – Charts (pure SVG, no external libraries):

Chart 1: Grouped bar chart – Monthly Cost + CPU per resource

Red bars for flagged, green for healthy
Chart 2: Donut chart – Savings split P0/P1/P2
Center shows total savings

Section 5 – Resource Inventory with:

- Waste Score (0-100) per resource
- Flagged vs Healthy status
- Full metrics per resource

Section 6 – Weekly trend horizontal bars (Week1 to Week4)

- Shows escalating waste over time
- Resource type waste distribution (EC2 vs RDS vs S3)

Section 7 – Interactive Savings Calculator:

- Drag slider to project monthly savings
- Recommendation coverage shown visually

Section 8 – Flagged resource cards with:

- Resource ID, type, region, CPU%, cost
- P0/P1/P2 severity badge color coded
- AI-generated recommendation
- Direct links to GitHub issue, Discord, email

Section 9 – Embedded AI Assistant chatbot:

- User can type questions like:
"Give me full summary"
"What are total savings?"
"Show all P0 critical issues"
- Chatbot answers based on actual scan data
- Powered by Ollama running locally

Dashboard auto-refreshes every 30 seconds.

Auto-opens in browser after generation.

Must work completely offline – no external JS libraries.

MAIN.PY – AGENT ORCHESTRATOR

1. ASCII banner at startup
2. Step-by-step progress with emojis (7 steps)
3. Final summary box with timing
4. CLI flags:
 - ask "question" → ask LLM a question and exit
 - watch [seconds] → live watch mode, auto-refresh
 - train → export AI training dataset JSONL
 - github → enable GitHub issues
 - discord → enable Discord alerts
 - email → enable email report
 - ai → enable Ollama AI analysis
 - serve → start local HTTP server
 - no-browser → skip auto-opening browser
5. Full graceful degradation:
 - Ollama offline → fallback rule analyzer, continue
 - GitHub fails → skip, warn, continue
 - Discord fails → skip, warn, continue
 - Email fails → skip, warn, continue
 - Dashboard ALWAYS generates no matter what fails

ADDITIONAL FEATURES

-
-
- Waste Score (0-100): composite score per resource weighing CPU%, cost, and idle days
 - Weekly trend analysis across all 4 weeks
 - AI Training Export: generates output/ai_training_dataset.jsonl with prompt/completion pairs for fine-tuning
 - Watch mode: monitors CSV file for changes and regenerates dashboard automatically

QUALITY REQUIREMENTS

1. Every file: proper docstrings and type hints
2. Every external call: try/except error handling
3. All credentials: in config.py only, never hardcoded
4. Terminal output: clean, formatted, with emojis
5. All SVG charts: hand-coded, no Chart.js, no D3
6. Code readable by a junior developer

TECH STACK

- Language: Python 3.10+
- LLM: Ollama llama3 (local, no API key needed)
- HTTP: requests library
- Config: python-dotenv
- Issues: GitHub REST API
- Alerts: Discord Webhooks
- Email: SMTP + smtplib
- Dashboard: Pure HTML/CSS/SVG

What the AI Generated From This Prompt

From this single prompt, the entire Cloud Cost Optimizer system was generated:

File	What Was Built
`data/cloud_billing_week1-4.csv`	4 weeks of realistic mock AWS billing data
`src/reader.py`	CSVReader class with waste score computation
`src/detector.py`	ResourceDetector with idle/oversized heuristics
`src/llm_analyzer.py`	Ollama integration with fallback rule engine
`src/github_issues.py`	Full GitHub Issues API automation
`src/discord_alert.py`	Discord webhook with rich embeds
`src/email_report.py`	HTML email via SMTP
`src/html_report.py`	Offline dashboard with SVG charts + AI chatbot
`main.py`	Agent orchestrator with CLI flags and graceful degradation
`config.py`	Centralized configuration
`requirements.txt`	Dependencies
`README.md`	Professional documentation

The Core LLM Prompt – Resource Analysis

This is the exact prompt template sent to Ollama for every flagged resource at runtime:

...

You are a cloud cost optimization expert working for a DevOps team. Analyze this cloud resource and provide a cost-saving recommendation.

Resource Details:

- ID: {resource_id}
- Type: {type} (EC2/RDS/S3)
- Region: {region}
- CPU Usage: {cpu_avg_percent}% (average over 30 days)
- Memory Usage: {memory_avg_percent}%
- Monthly Cost: \${cost_monthly_usd}
- Days Since Last Active: {last_active_days}

Your job:

1. Determine if this resource is idle or oversized
2. Explain WHY in 1-2 sentences (plain English)
3. Give ONE specific recommended action
4. Estimate monthly savings if action is taken
5. Rate severity: P0 (critical), P1 (warning), P2 (low)

Respond ONLY in this exact JSON format, nothing else:

```
{
  "severity": "P0",
  "reason": "explanation here",
  "recommendation": "specific action here",
  "estimated_saving": "$XXX/month",
  "confidence": "high"
}
```

...

Why This Prompt Works

Design Choice	Reason
JSON-only output	Feeds directly into GitHub, Discord, and dashboard without fragile string parsing
One recommended action	Keeps output actionable – maps directly to one GitHub issue title
Confidence field	Tells engineers when to verify manually before terminating a resource
Plain English reason	Non-technical stakeholders can read and understand the GitHub issue
All 7 resource fields	Every field contributes signal – CPU for idle detection, cost for financial impact, idle days for stale detection, type for termination procedure, region for compliance

Sample Input and Output

Input sent to Ollama

```json

```
{
 "resource_id": "db-02",
```

```

 "type": "RDS",
 "region": "eu-west-1",
 "cpu_avg_percent": 2.9,
 "memory_avg_percent": 24.8,
 "cost_monthly_usd": 295.00,
 "last_active_days": 45
}
...

```

### Output received from Ollama

```

```json
{
  "severity": "P0",
  "reason": "This RDS instance has been running at under 3% CPU for 45
days with minimal memory usage, indicating no active workload is using
it.",
  "recommendation": "Take a final snapshot and terminate this RDS
instance. Restore from snapshot if needed in future at a fraction of the
cost.",
  "estimated_saving": "$265/month",
  "confidence": "high"
}
...

```

What happens next with this output

1. `github\_issues.py` → files a GitHub Issue titled `[P0] db-02 is idle - save \$265/month`
2. `discord\_alert.py` → sends a red critical embed to the Discord channel immediately
3. `email\_report.py` → includes this finding in the HTML email report
4. `html\_report.py` → displays the P0 card on the dashboard with the AI recommendation

Fallback Strategy – When Ollama is Offline

If Ollama is not running, the agent automatically switches to a rule-based fallback:

```

```python
if cpu < 3 or saving > 200:
 severity = "P0"
elif cpu < 7 or saving > 100:
 severity = "P1"
else:
 severity = "P2"

reason = f"Resource has {cpu}% average CPU with a ${cost}/month bill."
recommendation = "Review utilization and consider termination or
downsizing."
confidence = "medium"
...

```

Terminal shows: `🚨 Ollama not available – using rule-based fallback analyzer`

All downstream steps – GitHub, Discord, email, dashboard – continue normally.  
The agent never crashes. Output is always produced.

---

## ## AI Assistant Chatbot Prompt (Dashboard)

The embedded dashboard chatbot uses this prompt at runtime:

```

You are an AI assistant embedded inside the Cloud Cost Optimizer dashboard.

You have access to the latest scan results shown below.

Current Scan Summary:

- Total resources scanned: {total_resources}
- Flagged resources: {flagged_count}
- Total estimated monthly savings: \${total_savings}
- Scan timestamp: {timestamp}

Flagged Resources:

{flagged_resources_json}

Answer the user's question based only on this scan data.

Be concise. Use bullet points where helpful.

If asked about a specific resource, include its ID, type, severity, and recommendation.

User question: {user_question}

```

## ### Example chatbot interactions

User asks	Chatbot responds with
"Give me full summary"	Total scanned, flagged count, savings, P0/P1/P2 breakdown
"What are total savings?"	Dollar amount + breakdown by severity
"Show P0 critical issues"	List of all P0 resources with IDs and recommendations
"Which resource wastes most?"	Highest cost flagged resource with details

---

## ## Prompt Version History

Version	Change	Purpose
v1.0	Initial resource analysis prompt	Basic idle/oversized detection
v1.1	Added severity guide inside prompt	Consistent P0/P1/P2 labels across runs
v1.2	Added "no markdown, no extra text"	Fixed JSON parsing failures



| v1.3 | Added confidence field | Engineers know when to verify before acting |  
| v2.0 | Added dashboard chatbot prompt | Interactive AI Q&A on scan results |

---

\*This document is maintained as a living record of all AI prompts used to build and operate Cloud Cost Optimizer. Any change to a prompt must be reflected here.\*